JavaScript'te `await` ifadesi teknik olarak kendi başına **sonsuz bir döngüye girmez**, ancak çözümlenmeyen (asla `resolve` veya `reject` edilmeyen) bir `Promise` bekliyorsa kodun yürütülmesini o noktada sonsuz bir şekilde askıya alır.

Bu durum gerçekleştiğinde JavaScript motorunda şunlar yaşanır:

- **Sadece İlgili Fonksiyon Durur:** `await` satırı, yalnızca içinde bulunduğu `async` fonksiyonun geri kalanının çalışmasını durdurur. Fonksiyonun dışındaki senkron kodlar ve diğer asenkron işlemler çalışmaya devam eder.
- **Tarayıcı veya Kilitlenme Yaşanmaz:** Klasik bir `while(true)` döngüsü gibi işlemciyi (CPU) %100 yük altına sokup **sayfayı veya uygulamayı kilitlemez**. Çünkü JavaScript olay döngüsü (Event Loop) boşa çıkar ve diğer görevleri işlemeye devam edebilir.
- **Bellek Sızıntısı (Memory Leak):** Askıda kalan `async` fonksiyon, çözülmeyen `Promise` ve bu fonksiyonun kapsamındaki (scope) tüm değişkenler bellekte (RAM) tutulmaya devam eder. Garbage Collector (Çöp Toplayıcı) bu verileri temizleyemez. Bu durum zamanla bellek sızıntısına yol açabilir.

🛑 Sonsuz Askıda Kalma Örneği

Aşağıdaki kodda `Promise` yapısı ne `resolve` ne de `reject` çağrısı yaptığı için `await` satırında kod sonsuza kadar bekler:

javascript

```
None
const aslaCozulmeyenPromise = () => new Promise(() => {}); //
resolve/reject çağrılmadı

async function Gorev() {
  console.log("Görev başladı...");

  // Kod burada sonsuza kadar askıda kalır.
  // Altındaki hiçbir satır çalışmaz.
  await aslaCozulmeyenPromise();

  console.log("Bu satır asla çalışmayacak!");
}

Gorev();
console.log("Ben senkron kodum, hemen çalışırım!"); // Bu çıktı
hemen ekrana basılır.
```

Kodu dikkatli kullanın.

🛠️ Bu Durum Nasıl Engellenir? (Zaman Aşımı Ekleme)

Asenkron işlemlerin (örneğin bir API isteğinin) sonsuza kadar askıda kalmasını önlemek için her zaman bir **Zaman Aşımı (Timeout)** mekanizması kullanmalısınız:

javascript

```
None
// Belirli bir süre sonra hata fırlatan yardımcı fonksiyon
const timeout = (ms) => new Promise((_, reject) =>
  setTimeout(() => reject(new Error("İşlem zaman aşımına uğradı!")),
ms)
);

async function veriGetir() {
  try {
    // API isteği ile 5 saniyelik zaman aşımını yarıştırıyoruz
    const veri = await Promise.race([
      fetch("https://ornek.com"),
      timeout(5000)
    ]);
    return await veri.json();
  } catch (hata) {
    console.error("Hata yakalandı:", hata.message);
  }
}
```

Kodu dikkatli kullanın.

Eğer kodunuzda belirli bir asenkron fonksiyonun ardından gelen işlemlerin durduğundan şüpheleniyorsanız, kodunuzu **zaman aşımı (timeout)** ile mi koruyorsunuz yoksa `fetch` **gibi işlemlerde** `AbortController` mı kullanıyorsunuz? Projenizdeki **spesifik senaryoyu** paylaşırsanız, en uygun koruma yöntemini birlikte kurgulayabiliriz.

Evet, NoSQL veritabanı bir dokümanı okumak veya güncellemek istediğinde, o dokümanın tamamını (tüm içeriğini) bir bütün olarak RAM'e yükler. NoSQL mimarisinin en temel kuralı budur: Doküman bölünemez bir bütündür.

İlişkisel (SQL) veritabanları ile NoSQL arasındaki bu farkı ve RAM ilişkisini şöyle özetleyebiliriz:

1. Parçalayarak Okuyamaz

SQL veritabanlarında (MySQL, PostgreSQL vb.) bir tabloda 100 tane sütun (column) olsa bile, siz sorguda `SELECT isim FROM kullanicilar` dediğinizde veritabanı RAM'e sadece o "isim" sütununu getirir. Satırın geri kalan büyük verisini RAM'e yüklemez.

NoSQL (örneğin MongoDB veya DynamoDB) sistemlerde ise durum farklıdır. Siz dokümanın içindeki tek bir alanı (field) çekmek isteseniz bile, veritabanı motoru **dokümanın tamamını diskten okur, RAM'e yükler, içinden istediğiniz alanı ayıklar ve size gönderir.**

2. Güncelleme (Update) Esnası

Dokümanın içindeki küçük bir ayarı değiştirmek istediğinizde de süreç aynıdır:

1. Dokümanın tamamı diskten **RAM'e çağrılır**.
2. RAM üzerinde o küçük değişiklik yapılır.
3. Doküman bir bütün olarak tekrar **diske yazılır**.

Eğer doküman boyutu 1 MB yerine 50 MB olsaydı, her küçük güncellemede 50 MB'lık devasa bir veri bloğu RAM'e yüklenecek, işlemciyi yoracak ve diske geri yazılacaktı. Bu da NoSQL'in o ünlü "milisaniyelik hız" avantajını tamamen yok ederdi.

3. Önbellekleme (Caching) Mantığı

Veritabanları performans için sık erişilen verileri RAM'de hazır tutar (Buffer Pool / Cache).

- Dokümanlar **1 MB** ile sınırlı olduğunda, veritabanı RAM'e **binlerce farklı kullanıcının** verisini sığdırabilir.
- Sınır olmasaydı ve birkaç kullanıcı 100'er MB'lık dokümanlar üretseydi, veritabanı RAM'i hemen dolacak ve diğer kullanıcıların istekleri diske kalacağı için sistem aşırı yavaşlayacaktı.

Özetle: Evet, doğrudan RAM'e yükler. Bu yüzden NoSQL tasarlarken dokümanları olabildiğince **küçük, hafif ve kompakt** tutmak hayati önem taşır.